

1 Lezione del 24-04-26

Concluso l'argomento del campionamento dei segnali, passiamo a discutere la *teoria dei codici* per quanto riguarda la parte della codifica dei segnali per poi trasmetterli su un mezzo.

1.1 Teoria dei codici

L'obiettivo della codifica di sorgente e di canale al trasmettitore è, rispettivamente:

- Per quanto riguarda la *codifica di sorgente* (**source encoder**), questa rimuove la ridondanza del segnale, per ottimizzare la quantità di informazione inviata (compressione);
- Per quanto riguarda la *codifica di canale* (**channel encoder**), questa introduce nuova ridondanza, sotto forma di meccanismi di protezione dagli errori.

In ricezione, i decoder ricostruiscono:

- Prima la parola di codice dal canale;
- Quindi il segnale originale dalla parola di codice (che codificava il segnale originale stesso).

Esistono 2 categorie principali di codici:

1. I codici a **rivelazione** di errore individuano la presenza di errori, senza correggerli;
2. I codici a **correzione** di errore riescono sia ad individuare che a correggere gli errori.

1.1.1 Codici di canale lineari

I *codici di canale lineari* sono particolari tipi di codici basati su combinazioni lineari di bit. Di questi ne esistono altre 2 categorie:

- Codici a **blocco**: operano su blocchi finiti di bit;
- Codici a **convoluzione**: operano su sequenze con memoria (quindi stream) di bit.

Il *rapporto* R di un codice a blocco è il rapporto fra numero di bit in ingresso k e numero di bit in uscita n per unità di trasmissione:

$$R = \frac{k}{n}$$

Idealmente vorremo un rapporto il più basso possibile, in quanto questo significherebbe che introdurremmo meno ridondanza possibile. Il rapporto di un codice è anche legato al tempo di trasmissione di una parola. Infatti varrà:

$$kT_b = nT_c$$

dove T_b è il tempo di ciascun bit di ingresso e T_c il tempo di ciascun bit di uscita. L'uguaglianza dovrà reggere affinché il sistema funzioni in tempo continuo, cioè si riescano a reintrodurre bit nel mezzo con la stessa frequenza con la quale si ottengono dal campionatore. Da questo seguirà che:

$$T_c = \frac{k}{n}T_b \leq T_b$$

cioè il tempo di trasmissione sul canale dovrà essere *minore* del tempo di ricezione di bit di ingresso. Riguardo alle frequenze R_c e R_b di uscita e di ingresso rispettivamente, varrà invece:

$$R_c = \frac{1}{T_c} = \frac{n}{k} \frac{1}{T_b} = \frac{n}{k} R_b \geq R_b$$

cioè in maniera analoga a prima, la frequenza di trasmissione sul canale dovrà essere *maggiore* della frequenza di ricezione di bit di ingresso.

1.1.2 Codici a blocco

Iniziamo a vedere 2 semplici esempi di codice a blocco.

1. Codice a ripetizione

Il codice a ripetizione è uno dei codici a blocco più semplici che esistano. Consiste nel riprodurre il bit che vogliamo trasmettere n volte sul canale di uscita. Ad esempio, un codice di ripetizione con $R = \frac{1}{3}$ mappa i bit direttamente a:

$$m = 0 \rightarrow c = [000], \quad m = 1 \rightarrow c = [111]$$

Potremmo quindi chiederci come decodificare il codice, tenendo presente che potrebbero esserci errori (bit-flip) lungo il segnale ricevuto. La risposta è scegliendo il bit più frequente nella parola ricevuta, cioè implementando la cosiddetta *decodifica a maggioranza*.

Un codice a ripetizione può *rivelare* fino ad $n - 1$ errori, in quanto si possono rivoltare fino a $n - 1$ prima di ottenere quella che è nuovamente una parola del codice (parole che contengono sia 0 che 1 sono automaticamente sbagliate per la definizione di codici a ripetizione).

Allo stesso modo, riesce a *correggere* fino a $\frac{n-1}{2}$ errori, in quanto quando oltre la metà dei bit viene rivoltata si può rilevare che si è verificato un errore, ma non verificare se si è risolto in maniera corretta.

Il tradeoff dei codici a ripetizione è che si spende molto più tempo a trasmettere lo stesso bit, in particolare con un fattore n . Nel caso in cui vogliamo trasmettere con le ipotesi di prima, cioè con la stessa frequenza di informazione all'ingresso e all'uscita sul canale, l'unica cosa che possiamo fare è aumentare la banda all'uscita, cioè su cui trasmettiamo il segnale.

Dal punto di vista probabilistico, data la probabilità di sbagliare un bit come p , la parola di sbagliare t bit in una parola di n bit è data dalla formula di Bernoulli:

$$p(t, n) = \binom{n}{t} p^t (1 - p)^{n-t}$$

dove:

$$\binom{n}{t} = \frac{n!}{t!(n-t)!}$$

Notiamo che nei sistemi di comunicazione reali la probabilità di sbagliare un bit è nell'ordine di $10^{-2} - 10^{-5}$. Siccome siamo capaci di correggere un errore di al massimo $\frac{n-1}{2}$ bit, la probabilità di errore della parola di codice è pari a:

$$P = \sum_{t=\frac{n+1}{2}}^n \binom{n}{t} p^t (1 - p)^{n-t} \approx \left(\frac{n}{\frac{n+1}{2}} \right) p^{\frac{n+1}{2}} (1 - p)^{\frac{n-1}{2}} \approx \left(\frac{n}{\frac{n+1}{2}} \right) p^{\frac{n+1}{2}}$$

dove facciamo l'approssimazione di porre la maggior probabilità sull'evento di errore più probabile.

2. Codice a controllo di parità

Consideriamo un altro semplice tipo di codice di rivelazione degli errori, con rapporto:

$$R = \frac{k}{k+1}$$

Abbiamo che di questi, usiamo:

- k bit di informazione effettivamente utile;
- $k+1$ bit di informazione inviata, di cui il bit in più è un bit di *parità* del blocco utile. Questo viene calcolato facendo la somma modulo 2 dei bit nella stringa e trascurando il riporto.

Senza bit di parità, un solo bit sbagliato fa sbagliare tutta la parola. Riprendendo la formula di probabilità si ha:

$$P = \sum_{t=1}^k \binom{k}{t} p^t (1-p)^{k-t}$$

che è piuttosto grande. Se aggiungo un bit di parità, la parola diventa di 12 bit, e si sbaglia quando si fanno almeno 2 errori. Gli errori di 1 bit vengono rivelati e corretti (mediante ritrasmissione). Notiamo che dal fatto che la maggior parte di errori si presentano come *spray*, la probabilità di un errore di 1 bit è circa del 50%. Riprendiamo quindi la formula di probabilità, stavolta:

$$P = \sum_{t=2}^k \binom{k}{t} p^t (1-p)^{k-t}$$

che è molto meglio.

1.1.3 Codici a blocco lineari

I codici a blocco **lineari** sono rappresentati da trasformazioni lineari di stringhe di k bit, dette *blocchi* $\mathbf{b}$:

$$\mathbf{b} = [b_1, \dots, b_k]$$

notiamo che definiamo questi blocchi come vettori **riga** (!). Definendo il codice a blocco in maniera tabulare, dove associamo ad ogni blocco di ingresso un blocco di uscita di lunghezza n , un codice a blocco lineare sarà l'*insieme* delle 2^k parole $\mathbf{c}$ di uscita generate dalla trasformazione lineare della stringa $\mathbf{b}$:

$$\mathbf{c} = [c_1, \dots, c_n] = \mathbf{b}G = \sum_{i=1}^k b_i \mathbf{g}_i$$

dove G è una matrice $k \times n$ detta *matrice generatrice* del codice. Le $\mathbf{g}_i$ a questo punto sono le k righe di dimensione n della matrice. Notiamo che anche $\mathbf{c}$ è un vettore *riga* (!!).

Le proprietà dei codici a blocco (facilmente verificabili) sono:

1. Ogni parola di codice è combinazione lineare delle righe di G ;

2. Il codice è costituito da tutte le possibili combinazioni delle righe di G ;
3. La somma di due parole di codice è ancora una parola di codice;
4. La n -upla di tutti zeri è sempre una parola di codice.

1.1.4 Distanza di Hamming

Definiamo *distanza di Hamming* fra due parole di codice $\mathbf{c}_1$ e $\mathbf{c}_2$ come il numero di posizioni di cui differiscono le parole di codice. La indichiamo come d_H :

1.1: Distanza di Hamming

Data:

$$d_H(\mathbf{c}_1, \mathbf{c}_2)$$

la d_H (distanza di Hamming) è il numero di posizioni di cui differiscono le parole di codice $\mathbf{c}_1$ e $\mathbf{c}_2$.

Il *peso di Hamming* di una parola di codice $\mathbf{c}$, a questo punto, è definito come la distanza dall'origine della parola:

$$w(\mathbf{c}) = d_H(\mathbf{c}, \mathbf{0})$$

In altre parole, il peso è il numero di bit a 0 di una parola $\mathbf{c}$.

A questo punto, la *distanza minima* di un codice è la minima distanza di Hamming fra tutte le parole di codice:

$$d_{min} = \min_{i,j} d_H(\mathbf{c}_i, \mathbf{c}_j) = \min_k d_H(\mathbf{c}_k, \mathbf{0}), \quad i \neq j$$